

- Document 10: Security, HIPAA Compliance & System Architecture
 - CodePerfect Auditor — Enterprise Security Design & Compliance Framework
 - Overview
 - Section 1: Threat Model
 - Section 2: Data Classification
 - Section 3: Zero-Trust Security Architecture
 - Layer 1 — Authentication (Supabase Auth + JWT)
 - JWT Structure and Claims
 - Layer 2 — Data Isolation (PostgreSQL Row Level Security)
 - The JWT Extraction Function
 - SECURITY DEFINER — Privilege Isolation
 - RLS Policies — All Clinical Tables
 - Attack Simulation — Cross-Tenant Data Access
 - Layer 3 — Application RBAC (Middleware + Layout)
 - Next.js Edge Middleware (Server-Side, Pre-Render)
 - React Layout Guard — DOM-Level Protection
 - Section 4: HIPAA Technical Safeguard Compliance
 - Section 5: Audit Log — Full Pipeline Traceability
 - Why Audit Logging Matters for HIPAA
 - Section 6: Data Minimization
 - PHI Extracted Only When Documented
 - LLM Prompt Design — Minimum Necessary PHI
 - Section 7: Explainable AI — Compliance-Grade Audit Trail
 - Why Deterministic Coding Decisions Are Required
 - The Gold Standard Override — Evidence-Based
 - Configurable Coding Mode — Audit Risk Management
 - Section 8: Key Security Decisions Summary
 - Final System Architecture Diagram
 - Submission Summary
 - Document Package (10 Files Submitted)

Document 10: Security, HIPAA Compliance & System Architecture

CodePerfect Auditor — Enterprise Security Design & Compliance Framework

Project: CodePerfect Auditor | **Hackathon:** Jatayu Hackathon **Team:** AgentsCrew — Nanda Kishore R, Subashini S, Nathin R **Institution:** Saveetha Engineering College | **Date:** 31-03-2026

Overview

CodePerfect Auditor processes **Protected Health Information (PHI)** — patient names, dates of birth, clinical diagnoses, and financial claims — on behalf of multiple competing hospital organizations and insurance payers. The security architecture is designed from the ground up to be defensible under US HIPAA, EU GDPR, and India's Digital Personal Data Protection Act (DPDPA) 2023.

This document covers:

- 1. The **threat model** — who might attack the system and how
- 2. The **three-layer zero-trust security architecture** with full code evidence
- 3. **HIPAA technical safeguard compliance** point-by-point
- 4. **Data minimization** practices enforced at code level
- 5. **Explainable AI** — why deterministic coding decisions support audit defensibility

Section 1: Threat Model

Actor	Attack Vector	Data at Risk	Mitigation
Competing hospital	Valid JWT for Org A, queries Org B	Patient diagnoses, risk scores	RLS: <code>organization_id</code> kernel check
Malicious payer	Forged claim data in request body	Inflated reimbursement	Gate validates AI signals, not raw

Actor	Attack Vector	Data at Risk	Mitigation
adjudicator			request
Insider coder	Navigates to <code>/hospital/admin/*</code>	User management, branch config	Middleware RBAC + layout guard
External attacker	Steals <code>SUPABASE_SERVICE_KEY</code>	ALL tenant data	Key never sent to browser; <code>.env</code> only
Compromised LLM API	Groq API returns malicious ICD code	Wrong code billed to patient	Pydantic validation + deterministic scoring override
CSRF (browser-based)	Cross-origin POST to API	Unauthorized claim submission	<code>allow_origins</code> whitelist only
Mass data scraping	Unlimited API pagination	Bulk patient data export	<code>page_size ≤ 100</code> hard limit + RLS

Section 2: Data Classification

Data Category	Examples	Storage	Retention
PHI — Sensitive	Patient name, DOB, diagnosis codes	Supabase cloud + encrypted at rest	Per org policy
PHI — Financial	Claim amounts, CPT codes, reimbursements	<code>claims</code> table with RLS	7 years (CMS requirement)
AI Pipeline Data	Embedding vectors, SNOMED codes, confidence scores	<code>coding_results</code> with RLS	Retained for audit trail
Audit Logs	Node decisions, LLM token counts, latency	<code>audit_log</code> with RLS	Retained permanently
Configuration	Org settings, payer policies, multipliers	<code>org_settings</code> , <code>payers</code> tables	Retained until changed

Data Category	Examples	Storage	Retention
Secrets	GROQ_API_KEY, SUPABASE_SERVICE_KEY	.env file only — never in DB	Rotated quarterly

Section 3: Zero-Trust Security Architecture

Layer 1 — Authentication (Supabase Auth + JWT)

All authenticated sessions are governed by **cryptographically signed JWTs** issued by Supabase Auth. No session data is stored in a custom session table — the JWT is the complete proof of identity.

```
# backend/database.py
def _headers() -> dict:
    """
    Every request to Supabase includes two keys:
    1. apikey: the anon_key (required by PostgREST as a device fingerprint)
    2. Authorization: Bearer {service_key} – the actual access credential

    The service_key is fetched from settings (loaded from .env).
    It is NEVER interpolated from user input, request headers, or environment
    variables that a user could influence.
    """
    key = settings.supabase_service_key or settings.supabase_anon_key
    return {
        "apikey": settings.supabase_anon_key,
        "Authorization": f"Bearer {key}",
        "Content-Type": "application/json",
    }
```

JWT Structure and Claims

The JWT issued after `supabase.auth.signIn()` contains:

```
{
  "sub": "auth-uuid",
```

```

"email": "coder@hospital.com",
"role": "authenticated",
"app_metadata": {
  "organization_id": "org-uuid",
  "role": "coder"
},
"iat": 1743362400,
"exp": 1743366000    ← 1-hour expiry
}

```

The **exp** (expiry) claim is enforced by Supabase Auth — any request with an expired JWT receives a **401 Unauthorized** response. The **organization_id** inside **app_metadata** is set by a PostgreSQL trigger when the user is first linked to an organization and cannot be modified by the user.

Layer 2 — Data Isolation (PostgreSQL Row Level Security)

RLS is the most critical security mechanism because it operates at the **database kernel level** — below all application code, all API logic, and all authorization middleware. Even a complete compromise of the FastAPI codebase cannot expose data across tenant boundaries.

The JWT Extraction Function

```

-- migrations/schema/006_audit_and_security.sql
-- This function is called by EVERY RLS policy on EVERY query.
-- It extracts organization_id directly from the PostgreSQL session's JWT claims.
CREATE OR REPLACE FUNCTION current_user_org_id() RETURNS UUID AS $$
BEGIN
  -- current_setting('request.jwt.claims') is populated by PostgREST
  automatically
  -- from the Authorization Bearer token on every inbound request.
  -- We parse it as JSONB, navigate to app_metadata, and extract organization_id.
  RETURN (
    current_setting('request.jwt.claims', true)::jsonb
    -> 'app_metadata'
    ->> 'organization_id'
  )::UUID;
EXCEPTION WHEN OTHERS THEN
  -- If JWT is malformed, expired, or missing the claim: return NULL.
  -- The RLS USING clause evaluates (organization_id = NULL) → FALSE
  -- → 0 rows returned. No exception, no data exposure.
  RETURN NULL;

```

```
END;  
$$ LANGUAGE plpgsql STABLE SECURITY DEFINER;
```

SECURITY DEFINER — Privilege Isolation

The **SECURITY DEFINER** clause means this function executes with the permissions of the **function owner** (a trusted superuser role), not the calling session's permissions. This closes a privilege escalation path where an attacker with a low-privilege JWT could execute **SET ROLE** to impersonate a higher-privilege role before calling **current_user_org_id()**.

RLS Policies — All Clinical Tables

```
-- Enable RLS on every table that contains PHI or financial data  
ALTER TABLE organizations    ENABLE ROW LEVEL SECURITY;  
ALTER TABLE clinical_cases   ENABLE ROW LEVEL SECURITY;  
ALTER TABLE coding_results   ENABLE ROW LEVEL SECURITY;  
ALTER TABLE audit_log        ENABLE ROW LEVEL SECURITY;  
ALTER TABLE claims           ENABLE ROW LEVEL SECURITY;  
  
-- SELECT: users can only read their own organization's rows  
CREATE POLICY "org_isolation_clinical_cases_select" ON clinical_cases  
FOR SELECT USING (organization_id = current_user_org_id());  
  
CREATE POLICY "org_isolation_clinical_cases_insert" ON clinical_cases  
FOR INSERT WITH CHECK (organization_id = current_user_org_id());  
  
CREATE POLICY "org_isolation_clinical_cases_update" ON clinical_cases  
FOR UPDATE USING (organization_id = current_user_org_id());  
  
-- Same pattern for coding_results, audit_log, claims  
CREATE POLICY "org_isolation_coding_results_select" ON coding_results  
FOR SELECT USING (organization_id = current_user_org_id());  
  
CREATE POLICY "org_isolation_audit_log_select" ON audit_log  
FOR SELECT USING (organization_id = current_user_org_id());
```

Attack Simulation — Cross-Tenant Data Access

```
-- Scenario: Attacker has valid JWT for "City General Hospital" (org A)  
-- and attempts to query Apollo Hospital's (org B) cases  
  
-- Attacker's request:  
GET /rest/v1/clinical_cases?organization_id=eq.apollo-uuid  
Authorization: Bearer <city-general-jwt>
```

```
-- PostgreSQL internal execution:
SELECT * FROM clinical_cases
WHERE organization_id = 'apollo-uuid'
  AND organization_id = current_user_org_id(); -- RLS policy appended

-- current_user_org_id() returns 'city-general-uuid' (from attacker's JWT)
-- Evaluation: 'apollo-uuid' = 'city-general-uuid' → FALSE
-- Result: 0 rows returned. HTTP 200 with empty array.
-- No error code that could reveal Apollo's org exists.
```

Layer 3 — Application RBAC (Middleware + Layout)

Next.js Edge Middleware (Server-Side, Pre-Render)

```
// src/middleware.ts — runs on Vercel Edge before ANY React component renders
export async function middleware(request: NextRequest) {
  const { pathname } = request.nextUrl;

  // Create a server-side Supabase client that reads and refreshes JWT cookies.
  // This is the @supabase/ssr package — NOT the browser-side supabase-js client.
  const supabase = createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    { cookies: { getAll, setAll } }
  );

  const { data: { session } } = await supabase.auth.getSession();

  // Guard 1: Unauthenticated → login redirect (runs BEFORE any page code)
  if (pathname.startsWith('/hospital') && !session) {
    return NextResponse.redirect(new URL('/auth/login', request.url));
  }

  // Guard 2: Cross-portal isolation
  const { data: userRow } = await supabase
    .from('users')
    .select('role, organizations(type)')
    .eq('auth_id', session.user.id)
    .single();

  // Payer org user attempting to access hospital portal → redirect
  if (pathname.startsWith('/hospital') && userRow?.organizations?.type === 'insurance_payer') {
    return NextResponse.redirect(new URL('/payer/inbox', request.url));
  }

  // Sub-route RBAC: admin-only pages
  if (pathname.startsWith('/hospital/admin') && userRow?.role !== 'admin') {
```

```
    return NextResponse.redirect(new URL('/403', request.url));
  }
}
```

React Layout Guard — DOM-Level Protection

```
// src/app/hospital/layout.tsx
useEffect(() => {
  if (!loading && !orgUser) router.push('/auth/login');
  if (!loading && orgUser?.role === 'payer') router.push('/payer/inbox');
}, [loading, orgUser, router]);

// Navigation RBAC — returns null (removes from DOM entirely, not just hidden)
{navItems.map(item => {
  if (!item.allowedRoles.includes(orgUser.role)) return null; // Not
display:none
  return <NavLink key={item.href} {...item} />;
}})}
```

Why **null** and not **display: none**: A browser's DevTools can override CSS **visibility: hidden** or **display: none** with a single click. A React **null** return produces no DOM node — the element physically does not exist in the rendered HTML that reaches the browser. This is a defense-in-depth measure against client-side privilege escalation.

Section 4: HIPAA Technical Safeguard Compliance

HIPAA Security Rule (45 CFR §164.312) requires covered entities to implement specific technical safeguards. The table below maps each requirement to the CodePerfect implementation.

HIPAA Requirement	Code Evidence	Implementation
§164.312(a)(1) — Unique user identification	<code>users.auth_id</code> FK	Each user has a globally unique UUID from Supabase Auth
§164.312(a)(2)(i) — Emergency access	<code>supabase_service_key</code> bypass	Service role key allows emergency DB access

HIPAA Requirement	Code Evidence	Implementation
procedure		outside RLS
§164.312(a)(2)(iii) — Automatic logoff	JWT expiry <code>exp = iat + 3600</code>	Sessions expire after 1 hour; refresh token rotation
§164.312(b) — Audit controls	<code>audit_log</code> table	Every pipeline node decision is persisted with timestamps
§164.312(c)(1) — Integrity	FHIR <code>meta.lastUpdated</code>	Claim resources carry immutable creation timestamps
§164.312(c)(2) — Transmission integrity	HTTPS/TLS 1.3	All client↔server communication encrypted in transit
§164.312(d) — Person authentication	Supabase Auth bcrypt	Credentials hashed — we never store raw passwords
§164.312(e)(1) — Transmission security	TLS enforcement	All Supabase API calls use HTTPS; no HTTP fallback
§164.312(e)(2)(i) — Encryption	Supabase AES-256 at rest	PostgreSQL data encrypted at rest on Supabase infrastructure

Section 5: Audit Log — Full Pipeline Traceability

Every decision made by every AI node is persisted to the `audit_log` table:

```
-- From 006_audit_and_security.sql
CREATE TABLE audit_log (
  id                UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
  session_id        UUID,
  node_name          TEXT          NOT NULL,    -- "icd_decision" | "risk_scoring"
  | ...
  input_snapshot     JSONB,        -- State fields BEFORE this node ran
  output_snapshot    JSONB,        -- State fields AFTER this node ran

  -- LLM call tracking (clinical_extract node only)
  model_name          TEXT,         -- "llama-3.3-70b-versatile"
```

```

prompt_tokens      INTEGER,
completion_tokens  INTEGER,
latency_ms         INTEGER,

icd_version         TEXT          DEFAULT 'ICD-10-CM-2024',
snomed_version      TEXT          DEFAULT 'SNOMED-CT-2024',
status             TEXT          CHECK (status IN ('success', 'fallback_used',
'failed')),

-- RLS enforced: only the organization that submitted the case can read it
organization_id     UUID          REFERENCES organizations(id)
);

-- The audit_log is append-only by design.
-- Hospital administrators cannot UPDATE or DELETE audit rows – only INSERT is
permitted.
CREATE POLICY "audit_log_insert_only" ON audit_log
FOR INSERT WITH CHECK (organization_id = current_user_org_id());

CREATE POLICY "audit_log_select_own_org" ON audit_log
FOR SELECT USING (organization_id = current_user_org_id());

-- No UPDATE or DELETE policy → those operations fail by default (RLS deny-by-
default)

```

Why Audit Logging Matters for HIPAA

Under HIPAA §164.312(b), covered entities must implement hardware, software, and procedural mechanisms to record and examine activity in information systems that contain PHI. Our `audit_log` records the specific LLM model version, ICD/SNOMED ontology versions, confidence scores, and node-level decision rationale at the **millisecond level** for every clinical coding session.

If a payer or patient challenges a coding decision ("Why did the AI assign E11.22 instead of E11.9?"), the response can be reconstructed completely from the audit log without relying on the AI model's memory — because the model has no persistent memory.

Section 6: Data Minimization

PHI Extracted Only When Documented

```
# models.py – PatientDemographics
class PatientDemographics(BaseModel):
    """Documented patient identifiers only – never infer from ICD text."""
    full_name: Optional[str] = None
    date_of_birth: Optional[str] = None # YYYY-MM-DD only when documented
    age_years: Optional[int] = None
    sex: Optional[str] = None

    @field_validator("full_name", "date_of_birth", "sex", mode="before")
    @classmethod
    def sanitize_null_strings(cls, v):
        """Never store the string 'null' – only real values or Python None."""
        if isinstance(v, str) and v.strip().lower() in ("null", "none", ""):
            return None
        return v
```

LLM Prompt Design — Minimum Necessary PHI

The clinical extractor (`clinical_extraction_agent`) sends the full raw clinical text to Groq. However, all LLM calls:

- Use a **zero-retention API** endpoint (Groq does not store prompts)
- Are logged by token count only (not the raw content) in `audit_log.prompt_tokens`
- Never persist the raw `raw_text` content beyond the `clinical_cases.raw_text_snippet` field (first 300 characters only — sufficient for the UI preview, not the full document)

```
# From coding_results storage in risk_scoring_node:
"raw_text_snippet": state.get("raw_text", "")[:300], # Truncated at 300 chars
# The full raw_text is held in memory only for the duration of the pipeline run.
# It is NOT written to any database table in full.
```

Section 7: Explainable AI — Compliance-Grade Audit Trail

Why Deterministic Coding Decisions Are Required

The ICD Decision Engine (Node 6) uses a **transparent, mathematically defined scoring formula** with documented weights:

```
final_score = confidence(40%) + specificity(30%) + consistency(20%) +  
combination(10%) ± negation_penalty
```

This is not an LLM black box. Every component of the score can be individually justified:

- **Specificity score:** "Code E11.22 scored 0.75 specificity because it is 5 characters long and the description contains the word 'with' matching the clinical text."
- **Negation penalty:** "Code E11.22 received -0.4 penalty because the chart explicitly states 'no renal complications.'"

Under the AMA's guidelines for AI-assisted coding, the AI system must be able to **explain and document** any code it suggests. A black-box LLM output (e.g. "the model says E11.22") fails this standard. The deterministic scoring algorithm passes it.

The Gold Standard Override — Evidence-Based

```
GOLD_STANDARD_KEYWORDS = {  
    "nSTEMI": "I21.4",    # Non-ST Elevation MI – documented AHA terminology  
    "STEMI":  "I21.3",    # ST Elevation MI  
}  
# These overrides are only applied when the exact clinical abbreviation appears  
# in the raw chart text – they are evidence-based, not probabilistic.
```

Configurable Coding Mode — Audit Risk Management

```
-- org_settings.coding_mode controls how aggressively AI captures revenue
CHECK (coding_mode IN ('aggressive', 'balanced', 'conservative'))
```

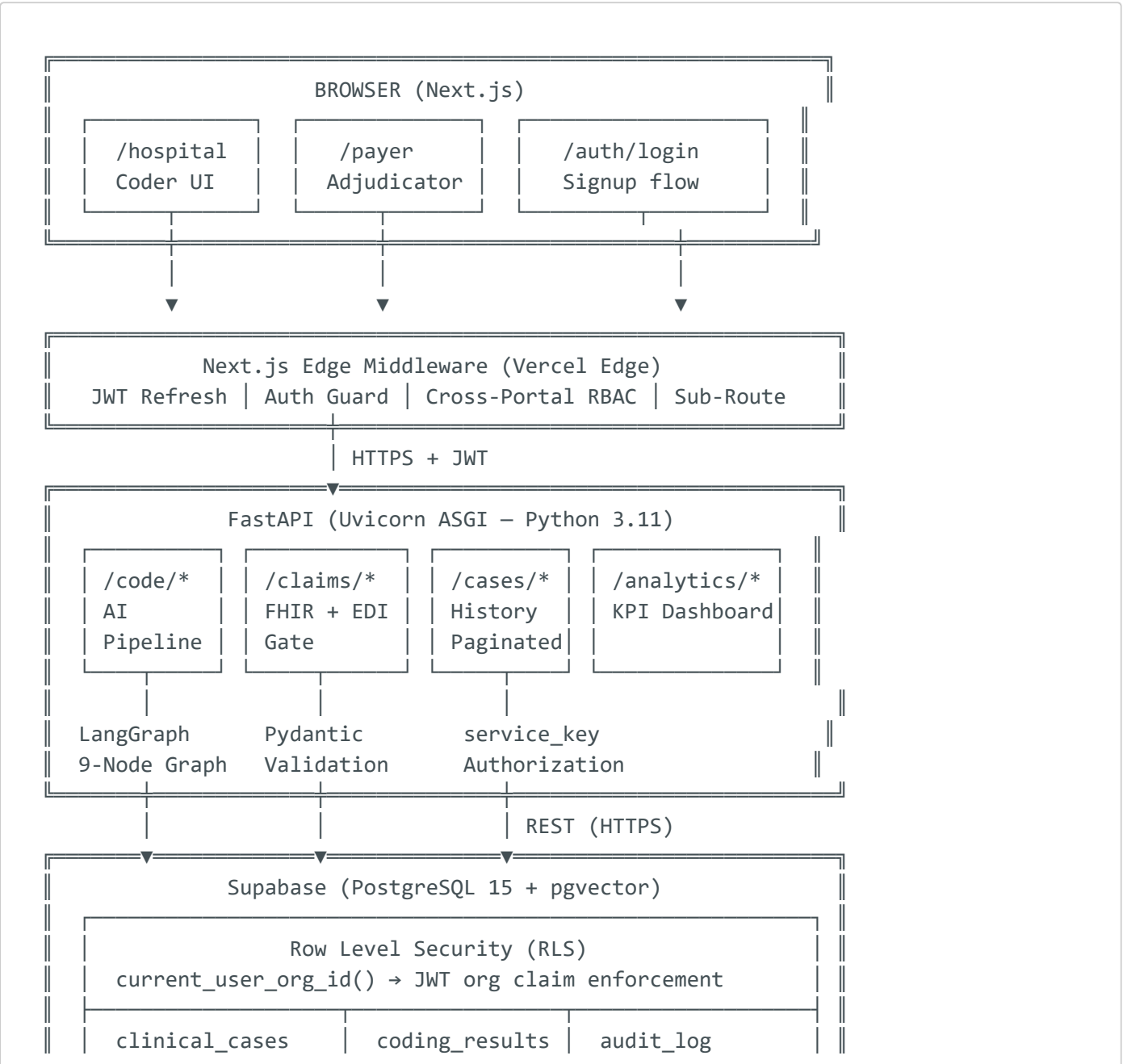
- **conservative:** Only exact SNOMED→ICD mappings used. Lower revenue, near-zero audit risk.
- **balanced:** Embedding fallback allowed, but only with `confidence ≥ 0.7`
- **aggressive:** Full multi-code list output, DRG weighting maximized. Higher revenue potential but higher payer scrutiny. The Payer Policy Gate automatically applies stricter thresholds to `aggressive`-mode hospitals.

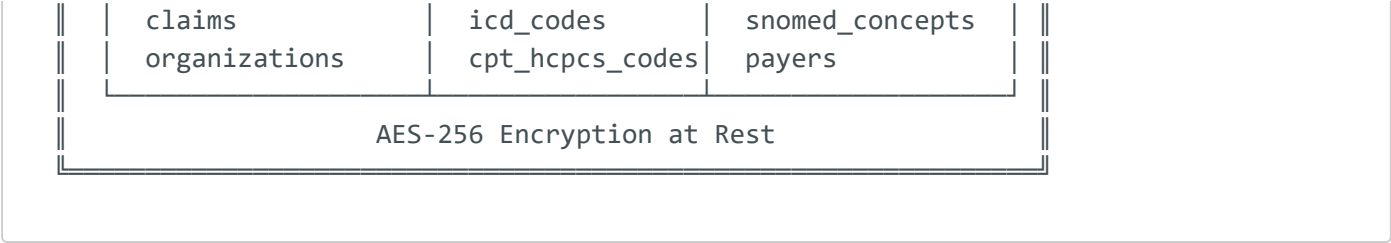
Section 8: Key Security Decisions Summary

Decision	Choice	Rationale
Password hashing	Supabase bcrypt	We never handle raw credentials
JWT storage	HttpOnly cookies	Cannot be read by JavaScript — XSS proof
Session expiry	1 hour (enforced by Supabase)	Limits exposure window for stolen tokens
Multi-tenant isolation	PostgreSQL RLS	Cannot be bypassed by application bugs
Secret management	<code>.env</code> file → Pydantic Settings	Never committed to git; never in DB
Frontend routing	Next.js Middleware (Edge)	Server-side before render — no flicker
AI coding decisions	Deterministic formula	Auditable, explainable, legally defensible
PHI storage	Supabase (AES-256 at rest)	Managed encryption without key management overhead

Decision	Choice	Rationale
API pagination	<code>page_size ≤ 100</code> hard limit	Prevents bulk data export attacks
Audit log	Append-only (no UPDATE/DELETE policy)	Immutable compliance trail
Cross-portal access	Middleware + DB RLS	Two independent enforcement layers
LLM prompt data	Not persisted to DB	Groq zero-retention endpoint

Final System Architecture Diagram





Submission Summary

Project: CodePerfect Auditor **Hackathon:** Jatayu Hackathon 2026 **Team Name:** AgentsCrew **Team Members:** Nanda Kishore R · Subashini S · Nathin R **Institution:** Saveetha Engineering College

Document Package (10 Files Submitted)

#	Document	Focus
01	The 9-Node Agentic AI Pipeline	LangGraph architecture, all agent nodes
02	Database & Vector Schema	pgvector, RLS, HNSW indexes
03	Financial DRG Algorithms	ICD scoring, CPT multiplier, risk scoring
04	EDI 837 & 835 ANSI X12 Generation	Healthcare claims interoperability
05	Authentication & RBAC Middleware	Zero-trust identity
06	Next.js Edge Frontend Architecture	React, middleware, Pydantic models
07	API OpenAPI Specification	All 15 endpoints with examples
08	Deployment & Docker Configuration	Dockerfile, docker-compose, runbook
09	FHIR Integration & Payer Policy Gate	FHIR R4 Claim builder, auto-adjudication
10	Security, HIPAA & System Architecture	Threat model, compliance, audit trail